Rapport d'Audit Complet

ESB-Learning Agentic AI Platform

Plateforme e-learning adaptative pour ESPRIT
École Supérieure Privée d'Ingénierie et de Technologies

Date :	16 Avril 2026
Auditeur :	GitHub Copilot (Analyse automatique)
Projet :	ESB-Learning v2.0
Score global :	**B+ (79/100)**
LLM principal :	Gemini 2.5-Flash / 2.5-Pro
Framework :	LangGraph + LangChain + Flask

4 Agents Agentic AI | 12 Skills | 21 MCP Tools | RAG Hybride 5 couches

Rapport généré automatiquement par analyse statique du code source

0 Table des matières

1 Vue d'Ensemble du Projet **2**
1.1 Architecture Générale . . . 2
1.2 Personas Utilisateurs . . . 2

2 Architectures Agentic AI **2**
2.1 Architecture 1 — ReAct Agent (Assistant Pédagogique) . . . 2
2.2 Architecture 2 — StateGraph + nœuds ReAct (Coach) . . . 3
2.3 Architecture 3 — StateGraph 11 nœuds hybrides (Exam Agent) . . . 3
2.4 Architecture 4 — Dual StateGraph (TP Agent) . . . 4

3 Agents et leurs Descriptions **4**
3.1 Agent 1 : Assistant Pédagogique . . . 4
3.2 Agent 2 : Coach Étudiant . . . 5
3.3 Agent 3 : Évaluateur d'Examens . . . 5
3.4 Agent 4 : Agent Travaux Pratiques . . . 5

4 Tools Disponibles **6**
4.1 MCP Tools — TP Agent (10 tools) . . . 6
4.2 MCP Tools — Exam Agent (11 tools) . . . 6
4.3 Tools LangChain (@tool) — Assistant (9 tools) . . . 7
4.4 Serveur MCP (JSON-RPC 2.0) . . . 7

5 Skills Disponibles **7**
5.1 Architecture du SkillManager . . . 7
5.2 Catalogue des 12 Skills . . . 8
5.3 Descriptions Détaillées . . . 8

6 Architecture Globale **9**
6.1 Vue d'ensemble du système . . . 9

7 Services Disponibles **9**
7.1 Services IA Principaux . . . 9
7.2 Services RAG / Documents . . . 10

7.3 Services Pédagogiques 10
7.4 Services Syllabus / Format Tunisie 10
7.5 Services Utilitaires 10

8 Techniques RAG Utilisées 11
8.1 Couche 1 — RAG Standard (ChromaDB + Dense Embeddings) 11
8.2 Couche 2 — CAG (Cached Augmented Generation) 11
8.3 Couche 3 — Hybrid RAG (Dense + Sparse) 11
8.4 Couche 4 — YouTube RAG (Multimodal) 12
8.5 Couche 5 — Multi-Format Processing 12
8.6 Résumé des Techniques RAG 12

9 Matrice Skills × Agents 12

10 Scores d'Audit et Recommandations 13
10.1 Scores Détaillés 13
10.2 Recommandations 13

1 Vue d'Ensemble du Projet

ESB-Learning est une plateforme d'e-learning adaptative pour ESPRIT (École Supérieure Privée d'Ingénierie et de Technologies, Tunis). Elle combine intelligence artificielle agentic, retrieval augmenté et gestion pédagogique complète.

1.1 Architecture Générale

Stack Technique

- **Backend :** Python 3.11+ / Flask 3.1.0 / SQLAlchemy 2.0 / PostgreSQL
- **Frontend :** Next.js 15 / TypeScript / React / Tailwind CSS
- **AI Framework :** LangGraph + LangChain + Google Gemini 2.5
- **Vector DB :** ChromaDB + FAISS + SentenceTransformer (all-MiniLM-L6-v2)
- **Protocole :** MCP (Model Context Protocol), JSON-RPC 2.0
- **NLP :** TunBERT (dialecte tunisien), BM25, transformers HuggingFace

1.2 Personas Utilisateurs

Rôle	Description	Agents accessibles
Étudiant	Apprentissage actif, quiz, TP	Assistant, Coach
Enseignant	Création examens/TP, analytics	Assistant, Exam, TP
Admin	Supervision globale	Tous les agents

2 Architectures Agentic AI

Le projet implémente **4 architectures Agentic AI distinctes**, chacune adaptée à son cas d'usage.

2.1 Architecture 1 — ReAct Agent (Assistant Pédagogique)

Agent Assistant — create_react_agent (LangGraph prebuilt)

- **Fichier :** `app/services/assistant_agent.py` (~893 lignes)
- **LLM :** Gemini 2.5-Flash, temperature=0.4
- **Pattern :** ReAct (Reason → Act → Observe)
- **Tools :** 9 base tools + N skills dynamiques
- **Trilinguisme :** FR / EN / Tunisien (Derja) via TunBERT

Boucle ReAct :

```
skill_tools = skill_manager.as_langchain_tools(
    agent_id='assistant', role=role, user_id=user_id)
all_tools = base_tools + skill_tools
agent = create_react_agent(llm, all_tools, prompt=system_prompt)
result = agent.invoke({"messages": messages})
# --> L'agent DECIDE autonomement quels tools/skills appeler
```

$$\begin{array}{ccc} \boxed{\text{User Message}} & \rightarrow & \boxed{\text{LLM : Think}} \\ & & \downarrow \\ \boxed{\text{Final Response}} & \leftarrow & \boxed{\text{Tool Call \& Observe}} \end{array}$$

2.2 Architecture 2 — StateGraph + nœuds ReAct (Coach)

Agent Coach — StateGraph 5 nœuds

- **Fichier :** `app/services/coach_agent.py`
- **Nœuds :** 1 déterministe + 3 ReAct sub-agents + 1 déterministe
- **State :** `CoachState` (TypedDict)

Pipeline :

$$\text{START} \rightarrow \underbrace{\text{collect_data}}_{\text{DB}} \rightarrow \underbrace{\text{analyze}}_{\text{ReAct}} \rightarrow \underbrace{\text{detect_gaps}}_{\text{ReAct}} \rightarrow \underbrace{\text{gen_plan}}_{\text{ReAct}} \rightarrow \underbrace{\text{finalize}}_{\text{assemblage}} \rightarrow \text{END}$$

2.3 Architecture 3 — StateGraph 11 nœuds hybrides (Exam Agent)

Exam Agent — StateGraph hybride

- **Fichier :** `app/services/exam_agent_graph.py` (~384 lignes)
- **Nœuds :** 5 nœuds ReAct + 6 nœuds déterministes
- **Bridge :** `mcp_langchain_bridge.py` (MCP → LangChain StructuredTools)
- **Output :** Analyse JSON + Document LaTeX + PDF compilé

```
extract_text -> extract_questions -> [classify_aa*] -> [classify_bloom*]
-> assess_difficulty -> compare_content -> [analyze_feedback*]
-> [suggest_adjustments*] -> [generate_corrections*]
-> generate_latex -> evaluate_proposal -> END
(* = noeuds ReAct autonomes)
```

2.4 Architecture 4 — Dual StateGraph (TP Agent)

TP Agent — 2 StateGraphs avec fallback

- **Fichier :** `app/services/tp_agent_graph.py` (~361 lignes)
- **Workflow Enseignant :** 5 nœuds (1 déterministe + 4 ReAct)
- **Workflow Étudiant :** 2 nœuds ReAct (correction + notation)
- **Langages :** Python, SQL, R, Java, C, C++
- **Fallback :** Chaque nœud ReAct a un fallback déterministe MCP

Workflow Enseignant :

$$\text{START} \rightarrow \underbrace{\text{get_context}}_{\text{det.}} \rightarrow \underbrace{\text{gen_statement}}_{\text{ReAct}} \rightarrow \underbrace{\text{parse_q.}}_{\text{ReAct}} \rightarrow \underbrace{\text{suggest_aa}}_{\text{ReAct}} \rightarrow \underbrace{\text{gen_ref}}_{\text{ReAct}} \rightarrow \text{END}$$

Workflow Étudiant :

$$\text{START} \rightarrow \underbrace{\text{auto_correct}}_{\text{ReAct}} \rightarrow \underbrace{\text{propose_grade}}_{\text{ReAct}} \rightarrow \text{END}$$

3 Agents et leurs Descriptions

ID	Nom	Type	Lignes	Pattern
`assistant`	Assistant Pédagogique	ReAct	~893	create_react_agent
`coach`	Coach Étudiant	StateGraph	~342	5 nœuds hybrides
`exam`	Évaluateur d'Examens	StateGraph	~384	11 nœuds hybrides
`tp`	Agent Travaux Pratiques	StateGraph	~361	Dual StateGraph

3.1 Agent 1 : Assistant Pédagogique

Tools de base (9) :

Tool	Rôle cible	Description
`get_my_courses`	Tous	Cours de l'utilisateur (étudiant/enseignant)
`get_calendar_activities`	Tous	Quiz, examens, séances, devoirs à venir
`get_course_details`	Tous	Chapitres, sections, documents d'un cours
`get_my_performance`	Étudiant	Scores quiz + analyse Bloom par niveau
`get_my_grades_summary`	Étudiant	Résumé notes par module
`get_recommendations`	Étudiant	Plan d'étude + exercices recommandés
`get_at_risk_students`	Enseignant	Étudiants à risque (scores/absences)
`get_class_performance`	Enseignant	Performance globale de la classe
`suggest_quiz_for_student`	Enseignant	Quiz ciblés sur faiblesses Bloom

Skills injectés dynamiquement via `SkillManager.as_langchain_tools()` selon rôle.

Feature TunBERT : Enrichissement dialecte tunisien — classification d'intentions NLP avant envoi au LLM.

3.2 Agent 2 : Coach Étudiant

Nœud	Type	Skills utilisés
`collect_data`	Déterministe	—
`analyze_performance`	ReAct	performance-scorer, bloom-classifier
`detect_gaps`	ReAct	weakness-detector, syllabus-mapper
`generate_plan`	ReAct	exercise-recommender, study-planner, feedback-writer, langua
`finalize`	Déterministe	—

3.3 Agent 3 : Évaluateur d'Examens

Pipeline complet 11 étapes :

Étape	Nœud	Type	MCP Tool	Skill
1	extract_text	Déterministe	extract_exam_text	—
2	extract_questions	Déterministe	extract_exam_questions	—
3	classify_aa	ReAct	classify_questions_aa	syllabus-mapper
4	classify_bloom	ReAct	classify_questions_bloom	bloom-classifier
5	assess_difficulty	Déterministe	assess_question_difficulty	—
6	compare_content	Déterministe	compare_module_vs_exam	—
7	analyze_feedback	ReAct	generate_exam_feedback	feedback-writer, rubr
8	suggest_adjustments	ReAct	suggest_exam_adjustments	syllabus-mapper, blo
9	generate_corrections	ReAct	generate_question_correction	rubric-builder, feedba
10	generate_latex	Déterministe	generate_exam_latex	—
11	evaluate_proposal	Déterministe	evaluate_exam_proposal	—

3.4 Agent 4 : Agent Travaux Pratiques

Workflow	Nœud	Type	MCP Tool	Skill
Enseignant	get_context	Déterministe	get_section_context	—
	generate_statement	ReAct	generate_tp_statement	quiz-generator
	parse_questions	ReAct	parse_tp_questions	bloom-classifier
	suggest_aa	ReAct	suggest_aa_codes	syllabus-mapper
	generate_reference	ReAct	generate_reference_solution	rubric-builder
Étudiant	auto_correct	ReAct	auto_correct_submission	code-reviewer

propose_grade	ReAct	propose_grade	feedback-writer

4 Tools Disponibles

4.1 MCP Tools — TP Agent (10 tools)

Fichier : `app/services/mcp_tools.py` — Registre : `MCP_TOOL_DEFINITIONS`

Tool	Description	Inputs requis
get_section_context	Récupère contenu pédagogique d'une section	section_id
generate_tp_statement	Génère énoncé TP dans le langage spécifié	context, language
suggest_aa_codes	Suggère codes AA pertinents pour un énoncé	section_id, statement
generate_reference_solution	Génère solution de référence + grille	statement, language
auto_correct_submission	Correction automatique code étudiant	statement, student_code, lan
propose_grade	Propose note numérique (0–max)	correction_report
parse_tp_questions	Parse énoncé et extrait questions/barème	statement, language
generate_question_starter	Génère template code étudiant	question_text, language
chat_with_student	Chatbot socratique (sans donner réponses)	question_text, student_mess
generate_correction_criteria	Génère critères de correction détaillés	statement, language

4.2 MCP Tools — Exam Agent (11 tools)

Fichier : `app/services/exam_mcp_tools.py` — Registre : `EXAM_MCP_TOOL_DEFINITIONS`

Tool	Description	Inputs requis
extract_exam_text	Extrait texte brut (PDF, DOCX, TXT)	file_path
extract_exam_questions	Parse et extrait questions structurées	exam_text
classify_questions_aa	Classifie par Apprentissages Attendus	questions, aa_list
classify_questions_bloom	Classifie par niveau de Bloom	questions
assess_question_difficulty	Évalue difficulté relative au cours	questions
compare_module_vs_exam	Compare syllabus vs examen	questions, aa_list
generate_exam_feedback	Génère feedback pédagogique détaillé	comparison_report, questions
suggest_exam_adjustments	Suggère ajustements équilibre/couverture	feedback, questions, aa_list

generate_exam_latex	Génère LaTeX + compile PDF	questions
evaluate_exam_proposal	Évalue proposition sur critères péd.	latex_source
generate_question_correction	Génère correction modèle	question_text, question_type,

Constantes pédagogiques :

- Niveaux Bloom : Mémoriser (10%) / Comprendre (20%) / Appliquer (30%) / Analyser (20%) / Évaluer (15%) / Créer (5%)
- Difficultés : Très facile / Facile / Moyen / Difficile / Très difficile
- Types questions : QCM, Ouvert, Pratique, Vrai/Faux, Calcul, Étude de cas

4.3 Tools LangChain (@tool) — Assistant (9 tools)

Voir tableau en section **??**.

4.4 Serveur MCP (JSON-RPC 2.0)

Fichier : `app/services/tp_mcp_server.py` Transport : `stdio` (JSON-RPC 2.0) **Status :** Implémenté mais non intégré en production.

5 Skills Disponibles

Le **SkillManager** orchestre **12 skills modulaires**.

5.1 Architecture du SkillManager

```
# Cycle de vie complet d'un skill
SkillContext(user_id, course_id, role, agent_id, params)
    -> SkillManager.execute(skill_id, context, input_data)
       |-- Validation acces (role check)
       |-- Course config override
       |-- SkillExecution tracking (DB)
       |-- Dependances resolution
       |-- _load_skill_function() -> module dynamique
       --> SkillResult(success, data, error, metadata)
```

Méthodes clés :

- `execute()` — Exécution d'un skill unique avec lifecycle complet
- `compose([ids])` — Chaîne séquentielle (output N → input N+1)
- `as_langchain_tools()` — Conversion skills → LangChain StructuredTools
- `resolve_for_agent(agent_id, role)` — Résolution dynamique
- `list_skills(agent_id, role, course_id)` — Liste filtrée

5.2 Catalogue des 12 Skills

ID	Nom	Catégorie	Agents	Temp.
bloom-classifier	Bloom Classifier	analysis	exam, tp, coach, assist.	0.1
syllabus-mapper	Syllabus Mapper	analysis	exam, tp, coach	0.2
feedback-writer	Feedback Writer	generation	exam, tp, coach, assist.	0.5
performance-scorer	Perf. Scorer	scoring	coach, assistant	0.1
weakness-detector	Gap Detector	analysis	coach, assistant	0.2
exercise-recommender	Exercise Rec.	generation	coach, assistant	0.4
study-planner	Study Planner	planning	coach	0.4
quiz-generator	Quiz Generator	generation	assistant, tp	0.5
content-summarizer	Summarizer	generation	assistant	0.4
code-reviewer	Code Reviewer	analysis	tp	0.3
rubric-builder	Rubric Builder	generation	exam, tp	0.3
language-adapter	Lang. Adapter	generation	assistant, coach	0.3

5.3 Descriptions Détaillées

`bloom-classifier` Classifie tout contenu selon les 6 niveaux de Bloom. Input : `content`, `content_type`. Output : `{bloom_level, confidence, justification}`. Le skill le plus réutilisé (4 agents).

`syllabus-mapper` Mappe le contenu aux Acquis d'Apprentissage (AA/CLO) du cours. Réservé aux rôles teacher/admin.

`feedback-writer` Génère un feedback pédagogique constructif et personnalisé. Temperature 0.5 pour créativité. Accessible à tous les rôles.

`performance-scorer` Calcule les scores de performance par AA, Bloom et module. Low temperature (0.1) pour précision.

`weakness-detector` Détecte et prioritise les lacunes par compétence. Utilisé par le Coach pour cibler les interventions.

`exercise-recommender` Suggère des exercices ciblés pour combler les lacunes identifiées.

`study-planner` Crée un planning hebdomadaire adaptatif. Réservé aux étudiants via le Coach.

`quiz-generator` Génère des questions de quiz alignées aux AA et niveaux de Bloom. Pour enseignants et admins.

`content-summarizer` Résume les chapitres adapté au niveau de l'étudiant. Assistant uniquement.

`code-reviewer` Review pédagogique de code étudiant. TP uniquement. Supporte Python, Java, C, C++.

`rubric-builder` Crée des grilles d'évaluation alignées aux objectifs. Pour Exam et TP.

`language-adapter` Adapte le registre de langue (FR/EN/Tunisien). Température 0.3.

6 Architecture Globale

6.1 Vue d'ensemble du système

```
+==============================================================+
|                    ESB-LEARNING PLATFORM                     |
+==============================================================+
|                                                              |
|  [ FRONTEND Next.js / TypeScript ]                           |
|  Pages: cours, quiz, TP, examens, chatbot, analytics         |
|              |  HTTP/REST API                                 |
|              v                                               |
|  [ BACKEND Flask (Python) ]                                  |
|  Routes: auth, courses, quiz, documents, ai, evaluate        |
|              |                                               |
|    +-----------+-----------+-----------+                     |
|    |           |           |           |                     |
|  [AGENT 1]  [AGENT 2]  [AGENT 3]  [AGENT 4]                  |
|  Assistant   Coach       Exam        TP                      |
|  ReAct       StateGraph  StateGraph  Dual StateGraph          |
|    |           |           |           |                     |
|    +-----+-----+-----+-----+           |                     |
|          |                             |                     |
|  [ SKILL MANAGER (12 skills) ]         |                     |
|  bloom | syllabus | feedback | perf    |                     |
|  gaps  | exercise | planner  | quiz    |                     |
|  summary | code | rubric | language   |                     |
|                                        |                     |
|  [ MCP TOOLS (21 tools) ]              |                     |
|  TP Tools (10) + Exam Tools (11)       |                     |
|  MCP-LangChain Bridge                  |                     |
|                                        |                     |
|  [ AI LAYER ]                          |                     |
|  Gemini 2.5-Flash / 2.5-Pro            |                     |
|  LangGraph + LangChain                 |                     |
|  TunBERT (NLP tunisien)                |                     |
|                                        |                     |
|  [ RAG PIPELINE ]                      |                     |
|  ChromaDB + all-MiniLM-L6-v2           |                     |
|  FAISS (dense) + BM25 (sparse)         |                     |
|  YouTube RAG (Gemini vision)           |                     |
|  CAG (section summaries)               |                     |
|                                        |                     |
|  [ DATABASE PostgreSQL ]               |                     |
|  Users, Courses, Quizzes, Exams        |                     |
|  Skills, SkillExecution, Enrollment    |                     |
+==============================================================+
```

7 Services Disponibles

7.1 Services IA Principaux

Service	Description
assistant_agent	Agent conversationnel trilingue ReAct
coach_agent	Coach performance + planning étudiant
exam_agent_graph	Évaluation et génération d'examens (11 étapes)
tp_agent_graph	Génération et correction de TP (dual workflow)

skill_manager	Orchestrateur centralisé des 12 skills
mcp_langchain_bridge	Bridge MCP tools → LangChain StructuredTools

7.2 Services RAG / Documents

Service	Description
vector_store	ChromaDB + SentenceTransformer (all-MiniLM-L6-v2)
document_pipeline	Pipeline complet traitement PDF (texte+images)
document_processor	Extraction texte, tableaux, images multi-format
youtube_rag_service	RAG YouTube (transcript + Gemini vision multimodal)
summarizer	Génération résumés sections pour CAG
smart_extraction_service	Extraction intelligente (PDF, DOCX, PPTX, TXT)

7.3 Services Pédagogiques

Service	Description
evaluate_service	Évaluation quiz et soumissions TP
feedback_service	Génération feedback formatif
practice_quiz_service	Quiz de pratique adaptatifs
exercise_extractor_agent	Extraction exercices depuis documents
chat_service	Service de chat pédagogique

7.4 Services Syllabus / Format Tunisie

Service	Description
syllabus_service	Gestion syllabus standard
syllabus_tn_service	Syllabus format Tunisie (AA/CLO)
tn_exam_evaluation_service	Évaluation examens format Ministère TN
tn_latex_report_service	Génération LaTeX/PDF rapports TN
aap_definitions	Définitions AAP (Activités d'Apprentissage)
program_extraction_service	Extraction programmes pédagogiques

7.5 Services Utilitaires

Service	Description

tunbert_service	NLP dialecte tunisien (TunBERT transformer)
piston_service	Exécution code multi-langages (sandbox)
video_service	Traitement vidéos pédagogiques
file_service	Gestion upload/download fichiers
mcp_tools	10 MCP tools pour les TP (JSON-RPC)
exam_mcp_tools	11 MCP tools pour les examens (JSON-RPC)
tp_mcp_server	Serveur MCP stdio (JSON-RPC 2.0)

8 Techniques RAG Utilisées

Le projet implémente une **architecture RAG hybride à 5 couches**.

8.1 Couche 1 — RAG Standard (ChromaDB + Dense Embeddings)

Vector Store — ChromaDB + all-MiniLM-L6-v2

- **Fichier :** `app/services/vector_store.py`
- **Modèle :** SentenceTransformer `all-MiniLM-L6-v2`
- **DB :** ChromaDB persistent (`instance/chroma_db`)
- **Collection :** une par document (`doc_{id}`)
- **Batch size :** 100 chunks par insertion

Types de contenu indexé :

- `text` — chunks de texte brut
- `summary` — résumés de sections (CAG)
- `image` — descriptions d'images/figures (AI-generated)
- `overview` — vue d'ensemble du document

8.2 Couche 2 — CAG (Cached Augmented Generation)

Les résumés de sections sont indexés séparément pour un accès rapide.

$$\text{context} = \underbrace{\text{summaries}}_{\text{CAG fast}} + \underbrace{\text{text_chunks}}_{\text{detail}} + \underbrace{\text{images}}_{\text{multimodal}} \quad (\text{max_chars} = 8000)$$

8.3 Couche 3 — Hybrid RAG (Dense + Sparse)

- **FAISS** (`faiss-cpu`) : Recherche vectorielle dense haute performance
- **BM25** (`rank-bm25`) : Recherche par mots-clés (TF-IDF sparse)
- **Fusion** : Les scores dense + sparse sont combinés pour une meilleure couverture

8.4 Couche 4 — YouTube RAG (Multimodal)

YouTube RAG — Fichier : `youtube_rag_service.py`

1. **Transcript** : `youtube-transcript-api` (SRT multi-langues FR/EN/AR)
2. **Vision** : Gemini 2.5 traitement natif URL YouTube (audio + frames)
3. **Fallback** : Analyse textuelle si vidéo native échoue
4. **Chunking** : CHUNK_SIZE=1400, CHUNK_OVERLAP=250
5. **Index** : ChromaDB (transcript + enrichissement Gemini + résumé + visuel)

Contenu extrait : `visual_summary`, `key_moments`, `visual_elements`, `topics_covered`, `pedagogical_summary`

8.5 Couche 5 — Multi-Format Processing

Formats supportés : PDF, DOCX, TXT, PPTX, images (vision LLM)
Librairies : `pdfplumber`, `pypdf`, `python-docx`, `camelot-py` (tableaux), `opencv`

8.6 Résumé des Techniques RAG

Technique	Technologie	Cas d'usage
Dense Retrieval	ChromaDB + all-MiniLM-L6-v2	Recherche sémantique documents
Sparse Retrieval	BM25 (rank-bm25)	Recherche mots-clés exacts
Fast Retrieval	CAG (summaries)	Vue d'ensemble rapide
Video RAG	YouTube API + Gemini Vision	Vidéos pédagogiques
Similarity Search	FAISS (faiss-cpu)	Index vectoriel haute perf.
Multi-type indexing	text + summary + image	Contenu multimodal unifié

9 Matrice Skills × Agents

Skill	Assist.	Coach	Exam	TP	Mode
bloom-classifier	✓	✓	✓	✓	Tool autonome (tous)
syllabus-mapper	—	✓	✓	✓	Teacher/Admin
feedback-writer	✓	✓	✓	✓	Tool autonome (tous)
performance-scorer	✓	✓	—	—	Coach-centric
weakness-detector	✓	✓	—	—	Coach-centric
exercise-recommender	✓	✓	—	—	Coach-centric
study-planner	—	✓	—	—	Student only
quiz-generator	✓	—	—	✓	Teacher/Admin
content-summarizer	✓	—	—	—	Student only
code-reviewer	—	—	—	✓	TP only

rubric-builder	—	—	✓	✓	Teacher/Admin
language-adapter	✓	✓	—	—	FR/EN/Tunisien

10 Scores d'Audit et Recommandations

10.1 Scores Détaillés

Critère	Score	Commentaire
MCP Tools définis	9/10	Schémas JSON complets (21 tools)
MCP Server production	4/10	Existe mais non intégré
Agent Assistant (ReAct)	10/10	ReAct parfait, autonomie totale
Agent Coach (StateGraph)	8/10	3 nœuds ReAct sur 5
Agent Exam (hybride)	7/10	5/11 nœuds ReAct, fallback robuste
Agent TP (dual)	7/10	ReAct + fallback déterministe
Skills comme Tools	8/10	`as_langchain_tools()` partout
State Management	9/10	TypedDict bien structurés
Execution Tracking	8/10	SkillExecution model complet
RAG Architecture	9/10	Pipeline hybride 5 couches
TOTAL	**79/100**	**B+ — Architecture hybride solide**

10.2 Recommandations

Remarque

Priorité Haute : Intégrer le serveur MCP en production (HTTP/SSE) et unifier `mcp_server.py` + `tp_mcp_server.py` pour couvrir les 21 tools.

1. **MCP Server en production** : Exposer le serveur JSON-RPC via HTTP/SSE pour interopérabilité avec d'autres clients MCP.
2. **Étendre syllabus-mapper à l'Assistant** : Actuellement limité à teacher/admin. Le rendre accessible aux étudiants.
3. **Unifier les serveurs MCP** : `mcp_server.py` + `tp_mcp_server.py` → un serveur unique couvrant les 21 tools.
4. **Dashboard Analytics SkillExecution** : Le modèle capture tout mais pas de UI. Créer `/admin/skills/analytics`.
5. **Cache LLM** : Utiliser Redis pour cacher les appels skills répétitifs (`bloom-classifier` sur le même contenu).
6. **Streaming SSE** : Les pipelines longs (Exam 11 nœuds) devraient streamer les résultats intermédiaires.
7. **Tests d'intégration agents** : Ajouter des tests automatisés pour les pipelines StateGraph complets.

Rapport généré par GitHub Copilot — Analyse automatique du code source ESB-Learning — 16 Avril 2026